ISO 20022 and Web APIs: An Implementation Best Practices White Paper

10 June 2025

By members of the ISO 20022 Registration Management Group and the Technical Support Group
Approved for publication by the ISO 20022 RMG 10 June 2025

This White Paper is a static document designed to provide an illustrative 'best practice' and to assist implementers in the financial services industry define Web APIs, *RESTful* Application Programming Interfaces (APIs) as a reference architecture style. The ISO 20022 RMG or its Technical Support Group do not accept liability for its use.

The document is illustrative and should be treated as such.

Feedback on this document including errata and suggestions for improvement are welcome by contacting:
ISO 20022 Technical Support Group
Any updates or revisions to this document will be made available here: ISO 20022 Whitepapers.
Secretariat: SIX Group (to be contacted via this page)

© 2025 – All rights reserved. ISO 20022 Registration Management Group

Contents

ISO 20022 AND WEB APIS: AN IMPLEMENTATION BEST PRACTICES WHITE PAPER I

CONTENTS II

FOREWORD II

INTRODUCTION III

NOTE: RMG RESOLUTION 17/383 PROPOSALS FROM TSG RELATING TO JSON AND APIS III

ISO 20022 AND WEB APIS 1

1 SCOPE 1
2 NORMATIVE REFERENCES 1
3 TERMS AND DEFINITIONS 2
4 WHAT IS REPRESENTATIONAL STATE TRANSFER (REST)? 3
4.1 INTRODUCTION 3
4.2 UNIFORM INTERFACE 3
4.3 STATELESS SESSIONS 4
5 ISO 20022 WEB APIS 5
5.1 INTRODUCTION 5
5.2 SCOPE LEVEL 6
5.3 CONCEPTUAL LEVEL 7
5.4 LOGICAL LEVEL 10
6 STYLE GUIDE 12
6.1 INTRODUCTION 12
6.2 OPERATIONS 12
6.3 THE RESPONSE 18
6.4 NAMING CONVENTIONS 23
7 VERSION CONTROL 25
8 API SECURITY 26
9 JSON SCHEMA TRANSFORMATION RULES 27
BIBLIOGRAPHY 27
10 REVISION HISTORY 28

Foreword

This document was prepared by a work group which included members of the ISO 20022 Registration Management Group and the Technical Support Group and was approved for publication by the full ISO 20022 RMG on 10 June 2025.

 © 2025 – All rights reserved. ISO 20022 Registration Management Group

Introduction

The purpose of this document is to help implementers in the financial services industry define *Web APIs*, based on new and existing models in the Repository, as defined by the international standard:

ISO20022:2013 Financial services — Universal financial industry information model and taxonomy.

This document was developed in response to a resolution by the ISO 20022 Registration Management Group (RMG), indicating worldwide demand for help in understanding the use of ISO 20022 where JSON is used as syntax.

Note: **RMG Resolution 17/383 Proposals from TSG relating to JSON and APIs**

The ISO 20022 RMG resolves to pursue the RESTful APIs and JSON initiatives through the TSG and the RMG secretariat will put out a call for interested parties to engage with that effort, with a view to delivering a more detailed report by July 18th 2017 that can form the basis of a Technical Report or Specification, to be published under the auspices of ISO, that would provide clear guidance as to how to produce standardised APIs that use the ISO 20022 repository artefacts.

This White Paper is a static paper that resulted from the work of the TSG. While it will inform the basis of a Technical Specification in ISO (see: ISO TC68/SC9/WG2), it is a separate document.

Since the publication of ISO20022:2013, exposing Web APIs based on the OpenAPI Specification has grown in popularity in the financial services industry. There are several alternative styles of JSON which could be used to create APIs based on the ISO 20022 information model. There are several competing specifications for the definition of Web APIs.

The document provides best practice suggestions on how APIs can be designed in a consistent way, reusing ISO 20022 repository artefacts that are common across services. These suggestions are provided in the hope that they will help decrease implementation time for developers that consume these APIs, ease mash-ups and foster reuse.

In order to enable the interoperability of web services in the financial industry, this document provides guidance on the generation of a Web API using OpenAPI as the specification language. As additional information to explain the choices made, this document may include a history of the alternatives considered, along with their explanations, the discussions and the final decisions.

Web APIs are a broad topic. The purpose of this White Paper is to:

- provide guidance relevant to the design of APIs in the context of the REST/JSON architecture style where possible,
- identify a common set of design principles and best practices that could be leveraged by the community to reduce fragmentation and increase interoperability when implementing an API,
- leverage open standard and RFCs so that the community has a common reference of key API design principles.

JSON is a widely used API data interchange format. OpenAPI is a popular description format that can be used for collaboration, documentation generation and comprehensive API description, such as security schema. The API documentation language should be OpenAPI Specification (OAS). OpenAPI has a very good ecosystem and offers many tools to support automation. Client software development kits (SDKs) for calling APIs from client applications, server-side scaffolding can be generated to implement the API services from an OpenAPI document. Mock servers can be generated to support exploration even before writing a single line of code. The same OpenAPI document can be used to automatically generate API documentation describing functionality and showing all the available resources and operations.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

Page Intentionally Left Blank

© 2025 – All rights reserved. ISO 20022 Registration Management Group

ISO 20022 and Web APIs

1 Scope

This document:

— provides best practice suggestions for modelling RESTful APIs with the artefacts currently embodied in the ISO 20022:2013 methodology,
— describes transformation rules from the ISO 20022 Logical Level to OpenAPI,
— but it does not address communication or storage security.

Note: It may also:

— identify work for further consideration in the next systematic review of ISO 20022;
— include references to recommended security and authentication standards such as the TC68/SC2 work items and the ISO/IEC JTC 1/SC 27 IT Security Techniques.

2 Normative references

ISO 20022-1, *Financial services — Universal financial industry message scheme — Part 1: Metamodel*

ISO 20022-3, *Financial services — Universal financial industry message scheme — Part 3: Modelling*

ISO 20022-4, *Financial services — Universal financial industry message scheme — Part 4: XML Schema Generation*

JSON Schema Specification Draft 2020-12 – JSON Schema - Specification [#section] (json-schema.org)

© 2025 – All rights reserved. ISO 20022 Registration Management Group

3 Terms and definitions

For the purposes of this document, the terms and definitions given in ISO 20022 part 1 and the following apply.

ISO and IEC maintain terminological databases for use in standardization at the following addresses:

— ISO Online browsing platform: available at http://www.iso.org/obp

— IEC Electropedia: available at http://www.electropedia.org/

3.0 Hypermedia

The concept of providing links to other resources. Hypermedia is one of the key principles of a REST architecture.

3.1 Remote procedure call (RPC)

An architecture style whereby operations are exposed to manipulate data through routines. This is done by putting the action in the URL (as opposed to REST).

3.2 Representation

A representation is information that is intended to reflect a past, current, or desired state of a given resource, in a format that can be readily communicated via the protocol.
(Source: RFC 9110: HTTP Semantics -rfc-editor.org)

3.3 Resource

The target of an HTTP request is called a "resource". HTTP does not limit the nature of a resource; it merely defines an interface that might be used to interact with resources. Most resources are identified by a Uniform Resource Identifier (URI).

3.4 REST

Representational State Transfer (REST) or RESTful web services**.**

© 2025 – All rights reserved. ISO 20022 Registration Management Group

4 What is representational state transfer (REST)?

4.1 Introduction

The REST architecture style is described by six constraints. The most relevant (to ISO 20022) are the **uniform interface** and the **stateless sessions**.

4.2 Uniform interface

4.2.1 Resource-based

Resources are identified in requests using URIs as resource identifiers. The resources themselves are conceptually separate from the representations that are exchanged with the client. The exchanged information is expressed in a particular syntax (e.g. XML or JSON), depending on the details of the request and the server implementation.

4.2.2 Manipulation of resources through representations

When a client knows a representation of a resource, including any metadata attached, it has enough information to modify or delete the resource on the server, provided it has the permission to do so.

4.2.3 Self-descriptive messages

Each message includes enough information to describe how to process the message. For example, which parser to invoke may be specified by an Internet media type (previously known as a MIME type). Responses also explicitly indicate their cache-ability.

4.2.4 Hypermedia as the engine of application state (HATEOAS)

Clients deliver state via body contents, query-string parameters, request headers and the requested URI (the resource name). Services deliver state to clients via body content, response codes and response headers. This is technically referred to as hypermedia (or hyperlinks within hypertext).

Aside from the description above, HATEOAS also means that, where necessary, links are contained in the returned body (or headers) to supply the URI for retrieval of the object itself or related objects.

In such architectures, each response would thus contain the subsequent representations in which the resource can be. So, the response to get to the “Authorised” state would also contain three new URIs, pointing to three states (Authorised, Rejected and Accepted).

Hypermedia may also be used to provide information about what the client is able to do next:

- pagination features
- further manipulations of resources or sub-resources or linked resources (relationships).

The uniform interface that any REST services must provide is fundamental to its design.

 © 2025 – All rights reserved. ISO 20022 Registration Management Group

4.3 Stateless sessions[1]

Stateless sessions indicate that the necessary state to process the request is contained within the request itself, whether as part of the URI, query-string parameters, body or headers. The URI uniquely identifies the resource and the body contains the state (or state change) of that resource. Then, after the server completes its processing, the relevant state or the necessary piece(s) of state are communicated back to the client via headers, status and the response body.

This is the opposite of a "session", which maintains state across multiple HTTP requests. In REST, the client must include all information for the server to fulfil the request, resending state as necessary if that state must span multiple requests.

Both the state and a resource are needed:

- State, or application state, is the data the server cares about to fulfil a request—the data necessary for the current session or request.
- A resource, or resource state, is the data that defines the resource representation—the data stored in the database, for instance. Application state can be considered as data which can vary from client to client and from request to request whereas resource state remains constant across every client which requests it.

[1] Information about sessions is not maintained between calls to the service. Instead, the URL provides enough information to access the same resource. However, the communication protocol or the application may maintain state.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

5 ISO 20022 Web APIs

5.1 Introduction

The premise for defining a modelling methodology for ISO 20022 REST APIs is to start from the current ISO 20022 methodology and extend it where necessary. Although the current methodology does not have custom REST API artefacts, many of the metamodel artefacts can be reused to design REST APIs.

This White Paper focuses on providing guidance for data driven financial services Web APIs (REST APIs). The rationale behind this choice is to provide guidance for a design-first approach to develop contracts based on the ISO 20022 information model and taxonomy.

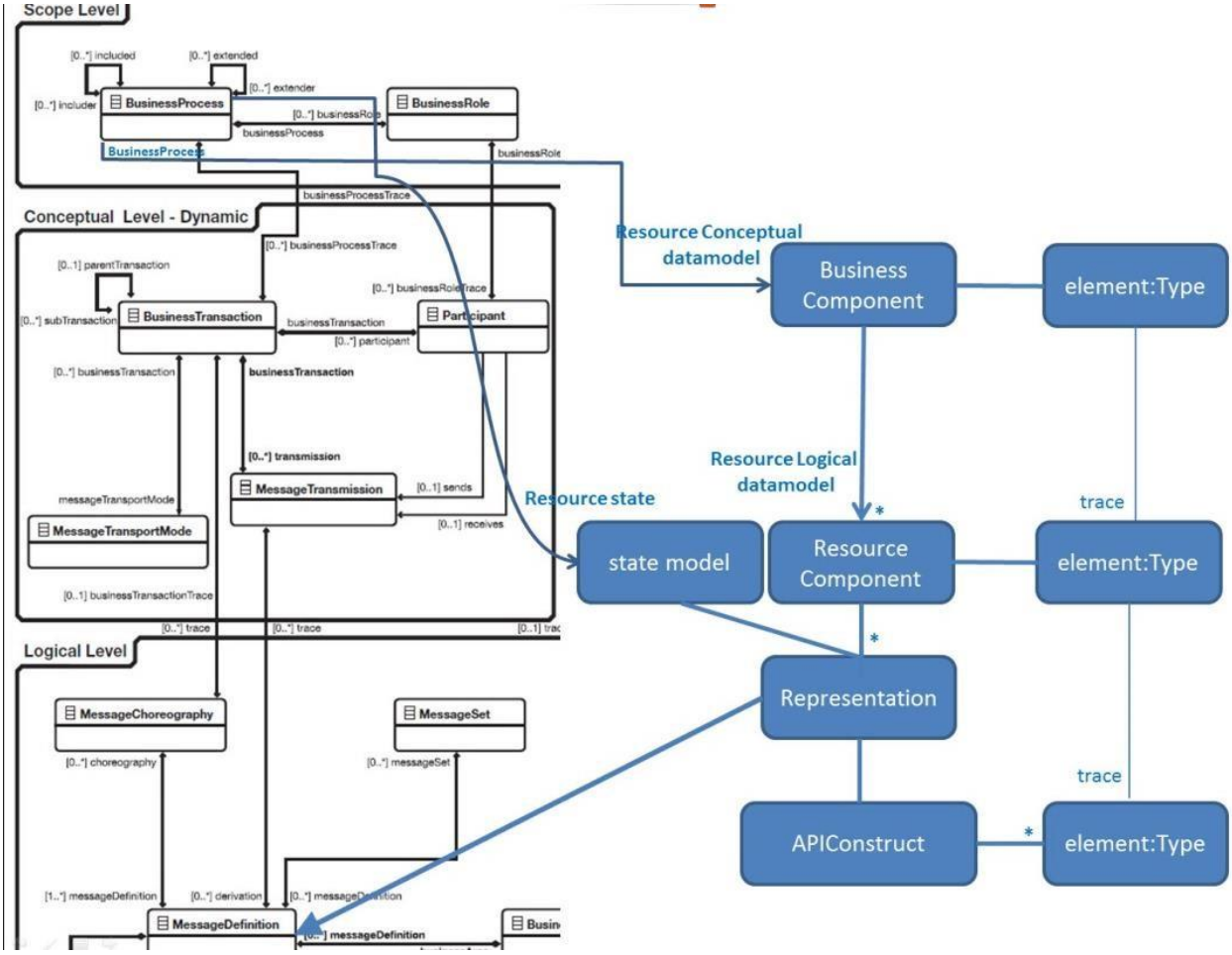


Figure 1 Leveraging components from the existing ISO 20022 methodology

© 2025 – All rights reserved. ISO 20022 Registration Management Group

5.2 Scope level

5.2.1 ISO 20022 metamodel

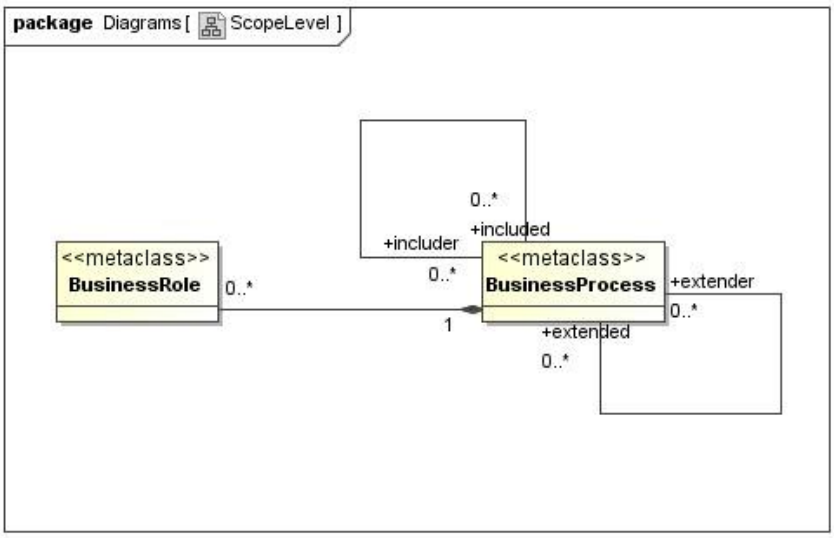


Figure 2 ISO 20022 Scope level metamodel

5.2.2 Modelling guidelines

The starting point is to analyse the business domain by specifying the Business Process and extracting the business concepts that are relevant to the business needs.

API design involves identifying the resources, their state transitions and modelling all the inputs and outputs. ISO 20022 defines a coherent collection of domain components within the broader business area or segment. The business components are the fundamental building blocks of an API.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

5.3 Conceptual level

5.3.1 ISO 20022 metamodel

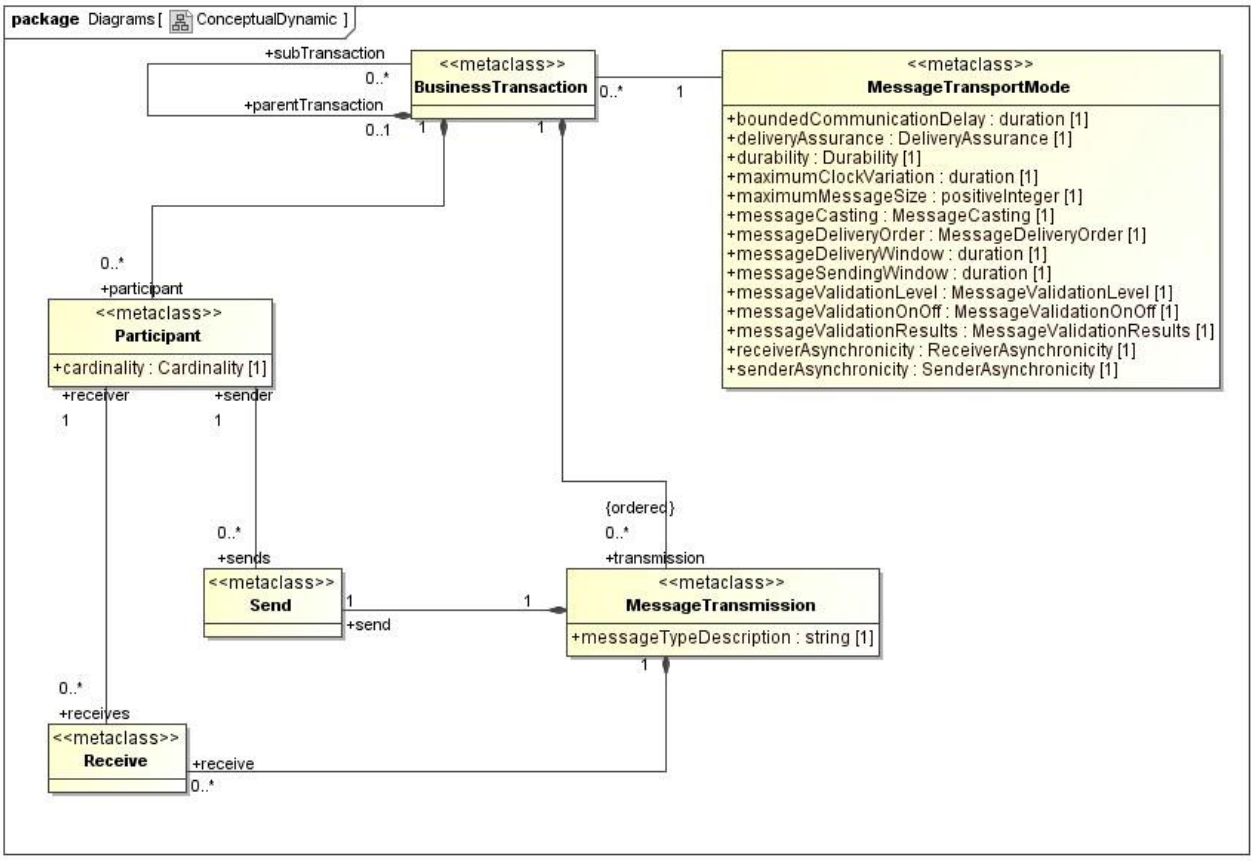


Figure 3 Conceptual dynamic Metamodel

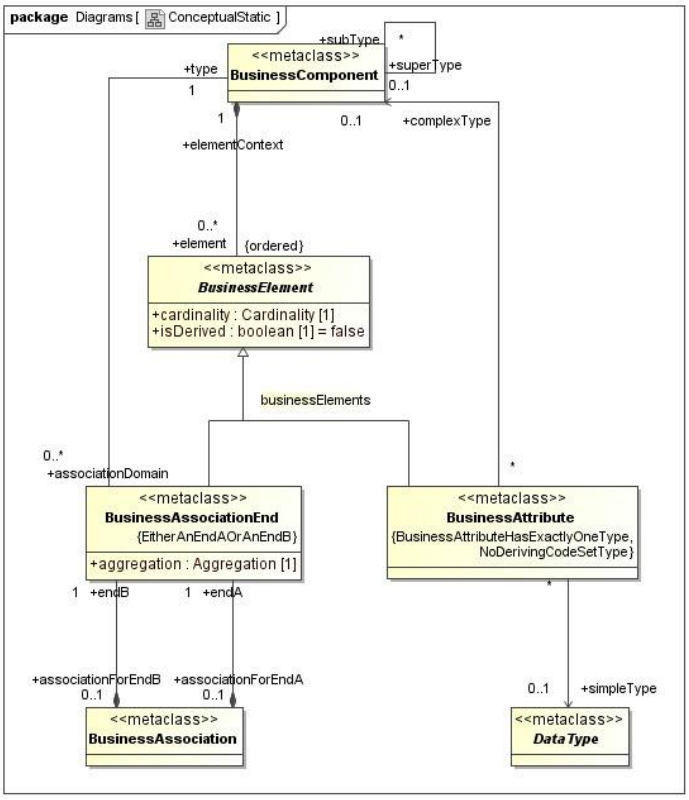


Figure 4 Conceptual Static Metamodel

© 2025 – All rights reserved. ISO 20022 Registration Management Group

A key aspect of modelling REST APIs is the ability to expose the different states that REST resources can have and the different methods that can be applied in each state.

The interface of the resource is a BusinessTransaction. The description of a Business Transaction in the metamodel and modelling allows it to describe an interface, whether to single or multiple resources and participants.

In essence, a BusinessTransaction provides for the grouping and sequencing of MessageTransmissions relevant to a Business Process. MessageTransmissions correspond to the API requests and responses.

5.3.2 Modelling guidelines

5.3.2.1 Modelling the resources (using Business Components)

The Business Components involved in the Business Process are either selected or created. The resulting data model is the basis for the various Resources that will be derived at the logical level in the context of a solution or a service.

The main purpose of the ISO 20022 model is to provide a model-driven development approach for defining APIs to access and process financial services data. When developing APIs, the business objects should be used where they meet the requirements or they should be extended to add new elements. For a given business purpose, the objects must be selected and bounded. Complex APIs may require a cluster of domain objects. In this case, an aggregate root should be identified and treated as a single unit. This aggregation is determined by the specific data needed to support the business function and rules. Typical aggregates include domain concepts such as Payment Instruction, Clearing Instruction and Settlement Instruction.

5.3.2.2 Modelling the states[2]

The different states are to be modelled by creating a state object for each state of a resource.

For each state, the different methods that can be applied, including their properties (safe, idempotent, etc.), are to be specified. As a state model represents the lifecycle of a resource, it may be necessary to provide different state models for different resources.

5.3.2.3 Designing the BusinessTransactions

A BusinessTransaction for a collection of resources specifies the **interactions** between a client and a server, corresponding to the Business Roles in the Business Process. The client sends MessageTransmission requests to the server and receives responses from the server. Thus, an interaction between two participants consists of a **request** and a **response**. One participant is the **supplier** (also known as the server) of the API service, the other the **consumer** (also known as the client) of the API service.

Chatty conversations that result in very long interactions are to be avoided.

[2] A state model is currently not part of the metamodel.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

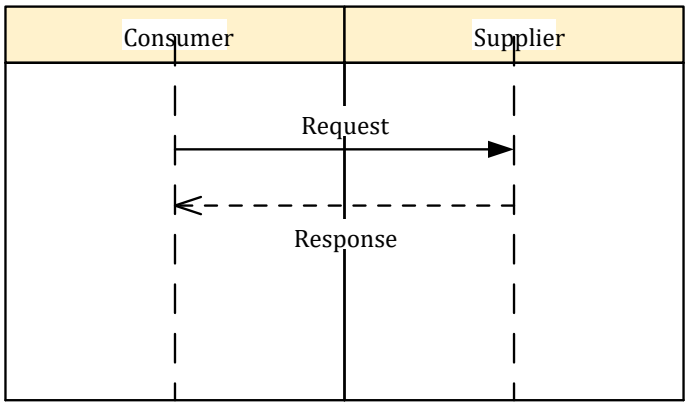


Figure 5 BusinessTransaction with one interaction

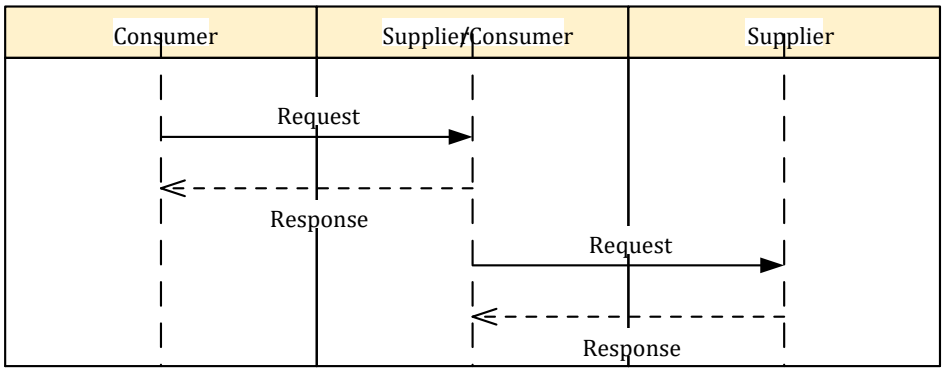


Figure 6 BusinessTransaction with two interactions

© 2025 – All rights reserved. ISO 20022 Registration Management Group

5.4 Logical level

5.4.1 Introduction

The purpose of the logical level is to define the resources and the methods that can be applied on each state of the resource(s). Resources are the foundation of any RESTful API design.

5.4.2 Metamodel

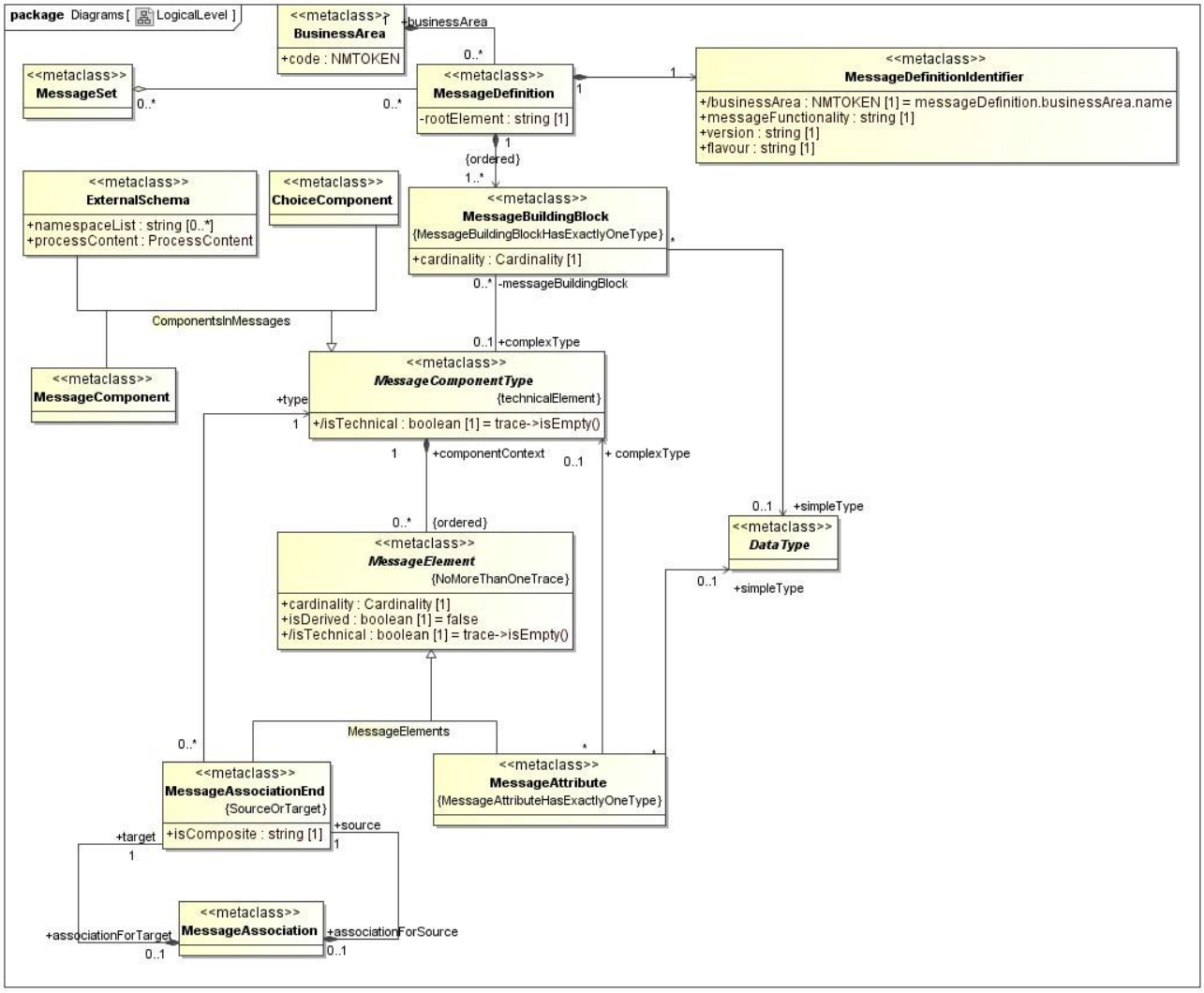


Figure 7 ISO 20022 Logical metamodel

The logical model consists of business concept definitions, a set of real-world concepts defined as business objects. The ISO 20022 model is a conceptual data model that captures a common business vocabulary. This provides a consistent definition of real-world business concepts, such as the Payment Instruction. In most cases, a type exists within the model and the information requirement are expressed as elements/attributes. Where a type cannot be identified within the models, a new object can be added.

Model driven design is about developing software that models real-world systems and processes. Before an API is designed, the process, business components and vocabulary of the domain in scope of the API are to be explored. The appropriate type in the model that describes the information requirement, expresses the details of a given business concept and its relationship to other concepts is to be identified. Once the business domain is understood, the aggregate root and cluster of objects are

© 2025 – All rights reserved. ISO 20022 Registration Management Group

to be identified. The elements that are required for the functioning of the API endpoint are to be identified. API definitions to support operations and all the input/output are to be created.

Traditional messaging approaches tend to define a data payload on a per-message basis. This means that each message defines an entirely new type to describe its data payload. This must be avoided in Web APIs.

5.4.3 Modelling guidelines

5.4.3.1 Refining the resource(s)

Resources are specified as a data model. All information represented by this model can be managed (created, read, updated and deleted) within an API solution.

Each resource is complemented by a state model defining the possible states of the resource and the applicable methods for each state.

A minimum of one state model must be defined, specifically the life-state model incorporating at a minimum the creation and the destruction states of a resource.

Resources may be linked to other resources through composition, aggregation or simple association.

Resources may be further refined to meet the requirements of a data provider.

Note

Resource identifiers may differ from the identifiers used in the business layer.

5.4.3.2 Designing the API Calls

5.4.3.2.1 Introduction

Each API call consists of a request message and a response message.

The response may be either

- a business response message (modelled as a representation response)
- an acknowledgment message with no response body
- or an error message (error structures are discussed in Section 6)

5.4.3.2.2 The representation

A representation specifies the data model of the method. The data model for the API call uses only elements from the resource data model. It may be supplemented with technical elements that are useful in the context of the method, such as page numbers (discussed in the Section 6).

A representation

- contains the data elements representing the state of a resource for a specific interaction. It is a precise description of the information that can be exchanged between two participants in the context of an interaction. Its data composition is similar to that of ISO 20022 messages, with the exception of the envelope element “Document”, which is missing in representations.
- has a very precise scope, which means that very little context needs to be added to the elements.
- is a composition of elements (MessageElements) that are collected from the resource(s) from which it is derived.

 © 2025 – All rights reserved. ISO 20022 Registration Management Group

5.4.3.2.3 Type and structure of request messages

The type of message describes what kind of operation needs to be processed on a given resource (see Section 6 for additional information on using the different HTTP methods).

The message must specify the chain of resources that are to be used (resource path). The path provided by the URL of the request lists the type and the identifier of each relevant resource.

Note

If an API has too many operations, this is an indication that either it was designed from an RPC perspective rather than using RESTful principles, or that the API in question is inherently better suited to an RPC type model.

6 Style guide

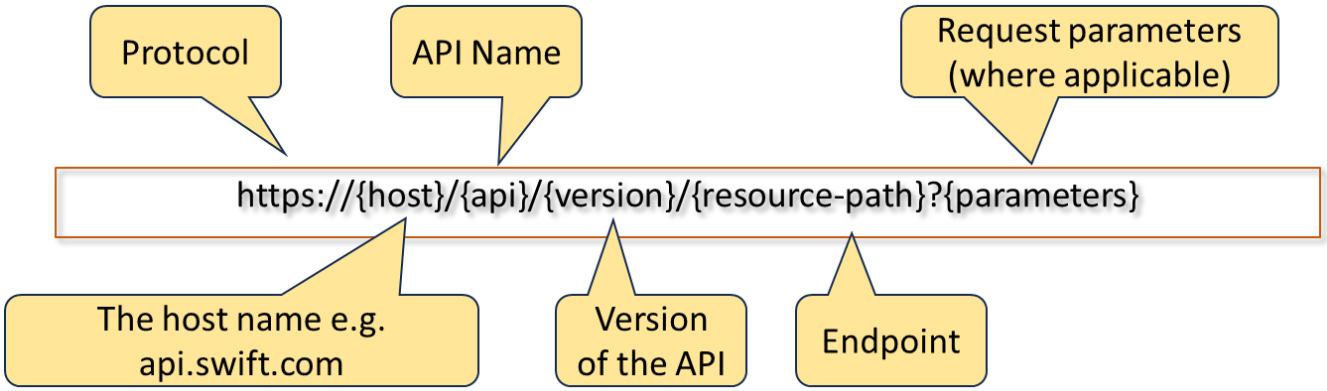
6.1 Introduction

This chapter provides guidance for Web API design at a high-level to ensure consistent API experiences, good usability and quality.

The syntax transformation adheres to the XML – JSON mapping rules specified in the Generation of JSON schema.

6.2 Operations

6.2.1 Endpoints



A resource is identified by a Uniform Resource Identifier (URI), which serves as its interaction interface. A logical structure for URIs improves the organisation of APIs, making them both easy to understand and consume. The first step in designing an API is to define the relative URL of each resource, which is also known as an endpoint. In OpenAPI, these endpoints are defined as path elements. When defining paths, it is advisable to choose a good name for each resource, which should not only be a noun (i.e., not a verb) but also intuitive – for example:

https://api.my-domain.com/cash-management/v1/accounts

6.2.2 Applying the standard HTTP methods

The HTTP specification defines methods, also known as HTTP verbs, for transferring or changing the state of a resource. The specification defines a set of standardised methods commonly used in HTTP and provides the semantics for the most common operations. When designing an API, it is advisable to adhere to the specification as developers are accustomed to this pattern. The HTTP protocol is well understood

© 2025 – All rights reserved. ISO 20022 Registration Management Group

by the development community as the protocol of choice for REST APIs. Any deviation or improper use leads to confusion, misrepresentation and implementation challenges.

It is not good practice in REST API design to equate HTTP methods with CRUD. There is no one-to-one correspondence. The only correspondences are that GET means read and DELETE really means delete. The specification permits multiple behaviours for POST and PUT – both may be used to create a resource.

6.2.2.1 Usage guidelines

The choice of method depends on the protocol selected for the implementation. The table below should be used as a guide for choosing a HTTP verb when HTTP is the protocol of choice. When deciding on the appropriate operation, please refer to the RFC for detailed guidance. The following is a high-level overview of HTTP methods and selection criteria as HTTP is the protocol of choice for Web APIs:

HTTP Verb	Operation	Description	Request Body	Safe	Idempotent	Cacheable
POST	Create	Create a new resource or query a resource collection.	Yes	No	No	Yes*
PUT	Create/Replace	Create (if the identifier for the resource is set by the client) or replace a resource.	Yes	No	Yes	No
PATCH	Partial Update	Partial update of an existing resource. See RFC 7396 or RFC 6902 for the two most common ways to implement a PATCH.	Yes	No	No	No
GET	Read/List	Retrieve an existing resource.	No	Yes	Yes	Yes
DELETE	Delete	Delete an existing resource.	No	No	Yes	No

* Depending on the implementation.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

Safe methods

An HTTP method is safe if it does not alter the state of the resource, that is, if the operation is read-only.

Idempotent methods

An HTTP method is idempotent if applying it to a resource multiple times produces the same state change of the resource as a single application (although responses may differ, particularly with a DELETE).

Cacheable methods

Responses may be stored for future reuse. As specified in RFC 7231, the POST method is cacheable in some cases, although most cache implementations only support the GET and HEAD methods.

6.2.2.2 Query parameter usage

Query parameters are typically used to filter, sort or paginate a collection of resources.

Filtering techniques are used to reduce the amount of data returned by the API, yielding:

- Lower bandwidth consumption
- Decreased latency
- Improved response time

Filtering and sorting are performed using resource attributes (e.g., region, year, month, day, etc.).

6.2.2.3 Content

Content has only one place where it belongs, and that is the request body, either as payload/body content or as a multipart/form-data request.

6.2.2.4 Collections

A collection resource consists of one or more item resource(s). An API consumer can retrieve an existing item resource by sending an HTTP GET request to the URI of that item resource. The "links" property in schemas is used to associate Link Description Objects with a resource instance. The value of this property must be an array, where each item in the array is a Link Description Object as defined in RFC 8288. There are other common approaches for links.

Link relation type

Links may include additional metadata through a link relation type, which identifies a link's semantics.

Collection objects can be extended with other members to build up rich functionality, such as link members that represent the collection object as an IRI-accessible collection resource:

rel – The value of this property must be a string. This property is required.

type – The value containing the HTTP verb used for the URL. If the method field is not included, then the link must be available using GET.

href – The href member identifies the target IRI (Internationalised Resource Identifiers) location of the resource.

Each resource should define a link to itself (IRI member) with a name equal to the case-sensitive octet sequence href. The reason for that is that a client should never have to track the link that returned a resource separately from the resource representation itself.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

Filtering a collection

Filters on a collection allow the client to get only the resources it is interested in from a collection.

When filtering is used, a GET request may provide criteria field based on the resource attributes (for example, colour, region, brand, date) to be added to the URL as query parameters.

Query parameters are specified in the form name=value. A query parameter list is appended to a URI after a "?". Multiple parameters are separated by "&".

Example

Use of logical operator OR (account-identification 12345678 or 87654321)

/cash-management/v1/accounts?account-identification=12345678,87654321

Use of logical operator AND (account-identification and currency)

/cash-management/v1/accounts?account-identification=12345678&currency=EUR

Pagination

APIs often need to provide collections of data, particularly those that are larger than a few hundred entries. However, collections may grow over time, increasing both lookup time and the size of the responses being sent over the wire. An API should support pagination from the outset, as adding pagination later constitutes a breaking change. Provider infrastructure, such as the API gateway or the provider server, imposes size limits, which pagination helps to overcome. Thus, collections must be paginated to limit the number of resources returned per request. Clients request pagination via query parameters, with pagination being represented as a JSON object.

Hypermedia links are of great value in navigating paginated resource collections. When returning paginated results, a service should support JSON-formatted responses that adhere to the following template and field descriptions. Additional members can be added to build up rich functionality.

Pagination should include the following fields for pagination links:

first: The link to the first page of resources
next: The link to the next page of resources
previous: The link to the previous page of resources
last: The link to the last page of resources

Note: When designing the pagination link, care should be taken to ensure that the return link relations are appropriate for traversing link relations. For example, there should not be a previous link on the first page, nor should there be a next link on the last page.

 © 2025 – All rights reserved. ISO 20022 Registration Management Group

For example, a Collection Resource with pagination:

GET /cash-management/v1/accounts?limit=25&offset=0


```
{
  "cash_accounts": [
    {
      "account_identification": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
.
. omitted for brevity
.
    }
  ],
  "links": [
    {
      "href": " https://developer.swift.com/cash-management/v1/accounts?limit=25&offset=0 ",
      "rel": "self",
      "type": "GET"
    },
    {
      "href": "https://developer.swift.com/cash-management/v1/accounts?limit=25&offset=100",
      "rel": "next",
      "type": "GET"
    },
    {
      "href": "https://developer.swift.com/cash-management/v1/accounts?limit=25&offset=75",
      "rel": "previous",
      "type": "GET"
    },
    {
      "href": "https://developer.swift.com/cash-management/v1/accounts?limit=25&offset=200",
      "rel": "last",
      "type": "GET"
    }
  ]
}
```


The example above shows support for pagination information through an implementation called “Hypermedia link relations”. Each element in the “Links” object has a hash called “rel”, which may contain information about first, next, previous and last depending on the size of the result. These relations also contain information about the resulting URL.

To make the results easier to handle and to decrease latency and lower bandwidth consumption, the results should be paginated.

If the client does not specify limit and offset, the service implementation must choose an appropriate default, which the API should document. The API must not return an error.

Limit – The maximum number of items to return in a response. If not specified, the default limit is returned.

Offset – The numeric offset identifies the page token that allows users to advance to the next page in the collection. The value 0 (zero) identifies the first page of the entry.

Note: The above is an example of offset pagination. The type of pagination supported by an API is an implementation choice.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

Empty collection

A collection contains a list of resources of the same type. If a request to the collection endpoint result in no match, the endpoint will return an empty array. The empty list is not an error since, in this case; the resource is the list itself which is being accessed.

For example:

GET /cash-management/v1/accounts?limit=25&offset=0

```
{
  "cash_account": []
}
```


© 2025 – All rights reserved. ISO 20022 Registration Management Group

6.3 The response

6.3.1 Object and status

All responses must have appropriate HTTP headers, response code to communicate the result of a request and MUST adhere to its semantic as defined in HTTP protocols RFCs.

6.3.2 Success response

All success responses must have an appropriate 2XX HTTP response code. The success response to a request is an HTTP response with HTTP status code *200 – "OK",* followed by a response body containing the (potentially partial) representation of the resource and its state depending on the request method. Responses for status code 204 must not have a response body.

6.3.3 Error response

All operations must have a HTTP 4XX and 5XX error response and should have a "default" (error) response.

Standardised HTTP status codes must be used consistently with their intended semantics as defined by RFC 7231. However, that may not be always sufficient to convey application-specific errors. A custom schema must be used to return more information about an error in the error response body. RFC 7807 defines an (extensible) problem details object that can be used with the media type "application/problem+json". For instance:

```
{
 "title": "Not Found",
 "status": 404,
 "detail": "ISIN 'US012071998' does not exist."
}
```

Fields	M/O	Description
type	O	A URI reference (RFC 3986) describing the problem type (in the context of the provided API). Should resolve to a human-readable document. "about:blank" is assumed if none is provided.
title	O	A short, human-readable summary of the problem type. It should not change from occurrence to occurrence of the problem.
status	M	The HTTP status code (see RFC 7231, Section 6) generated by the origin server for this occurrence of the problem.
detail	O	A human-readable explanation specific to this occurrence of the problem.
instance	O	A URI reference identifying the specific occurrence of the problem. It may or may not yield further information if dereferenced.

RFC 7807 is extensible. For instance:

```
{
 "type": "https://example.net/validation-error",
 "title": "Your request parameters didn't validate.",
 "invalid-params":
   [
    {
     "name": "age",
     "reason": "must be a positive integer"
    },
```

© 2025 – All rights reserved. ISO 20022 Registration Management Group

```
    {
      "name": "color",
      "reason": "must be 'green', 'red' or 'blue'"
    }
  ]
}
```

Another approach can be to return an array of error objects instead of only one.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

6.3.4 HTTP status codes

Depending on the type of response, the API must use the appropriate HTTP status codes to communicate the result. Standard HTTP status codes should be used; please consult the RFC 7231 status code definitions for more information. Use the tables below as guidelines to formulate the response.

Success

HTTP Status Code	Formal Description	Usage Guidelines
200	OK	The request has been completed successfully. The response depends on the request method used (e. g. GET, POST, etc.). For GET, POST, PUT or PATCH, a representation describing the current state and guiding the recipient's future interactions should be included.
201	Created	The request has been fulfilled successfully. Resource created. The HTTP response should contain a Location HTTP header with the URI of the newly created resource.
202	Accepted	The request has been accepted for processing and the action will likely succeed but has not yet been enacted. HTTP lacks a facility for sending a status code from an asynchronous operation. Typically used for long running processes. The code can be used to inform the consumer that the request has been received but will be fulfilled later.
204	No Content	The request has been fulfilled successfully and no further information is to be provided.

Client errors

The 4XX- class status codes indicate that problems originating in the client request and that the server is unable to process the request. The client should not resubmit the request unmodified.

HTTP Status Code	Formal Description	Usage Guidelines
400	Bad Request	The server cannot or will not process the request due to a client error in the request (e.g. malformed request syntax).
401	Unauthorised	This really means "Unauthenticated" and the request lacks valid authentication credentials. The response must include a WWW-Authenticate header field with a challenge applicable to the requested resource.
403	Forbidden	This really means "Unauthorised". The request is valid, but the server is refusing action because the client either does not have the necessary permissions for the resource or needs an account. At this point, access is permanently forbidden, and an action is required before the client can resend the request to this URL.
404	Not Found	The requested resource cannot be found, but may be available in the future. Subsequent requests from the client are permissible.
405	Method Not Allowed	The HTTP method used to access this resource is not supported by the provider. The response must include an Allow header field listing the methods/verbs supported by the target resource.
406	Not Acceptable	The server is not able to generate any of the client's preferred media types specified in the Accept request header. Clients indicate the list of accepted media types in the Accept HTTP request header field (for example, Accept: application/json, application/xml).
409	Conflict	The request cannot be completed due to a conflict with the current state of the target resource. Conflicts usually occur in response to PUT/PATCH requests when versioning (based on ETags) is used and

© 2025 – All rights reserved. ISO 20022 Registration Management Group

		a previous request has modified the state on the server, or a stale representation is used.
412	Precondition Failed	The status code indicates that one of the preconditions that the requester provided evaluated to false. See RFC 9110 for more details on request headers that can be used to apply a condition to a request.
413	Content Too Large	The server is unable to process the request because the data sent in the request is larger than the server can handle. To resolve this issue, please reduce the size of the upload or contact the server administrator to inquire about potential size limit adjustments.
414	URI Too Long	The status code indicates that the URI requested by the client is longer than the server is willing to interpret.
415	Unsupported Media Type	The status code indicates that the media type is not supported by the method or that the resource does not support it as requested by the client.
429	Too Many Requests	The status code indicates that the user has sent too many requests in a given amount of time ("rate limiting"). Use with Rate Limit headers.

Server errors

The 5XX- class status codes indicate that the server has encountered an unexpected condition.

HTTP Status Code	Formal Description	Usage Guidelines
500	Internal Server Error	The status code indicates a generic error message encountered due to an unexpected condition encountered by the server.
502	Bad Gateway	The status code indicates that the server, while acting as a gateway to get the response needed to process the request, received an invalid response.
503	Service Unavailable	The server is not ready to process the request, either because it is down for scheduled maintenance or because it is overloaded. The HTTP response should contain the Retry-After HTTP header with the estimated time for the service to be restored.
504	Gateway Timeout	The status code indicates that a server, acting as a proxy/gateway, did not receive a response from the upstream server within a specified time.

6.3.5 HTTP headers

HTTP requests and HTTP responses have an HTTP header section.

HTTP headers enable HTTP clients and HTTP servers to communicate with each other (for example, allowing a server to inform a client that it needs to provide authentication details to access a protected resource).

Header field names are case-insensitive.

Apart from that, there are no clear rules, standards or RFCs define naming conventions for HTTP headers.

The most common HTTP headers are described below.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

Standard request headers

Use the Accept and Content-Type Headers

The API should be built from the start with the ability to add content types in the future. This is because as the requirements evolve, the API may need to support a different security or hyper-media implementation. Using Content-Type headers provides the flexibility to support multiple types from the outset without worrying about breaking backwards compatibility.

A media type (MIME type) is a label used to identify a type of data supported by the server or consumer. The media type of a response or a request body must be declared for PUT, POST and PATCH in the OAS (using the OAS tag "content") For instance for a response:

```
/accounts:
  get:
    description: Returns all accounts from the system to which the user has access
    responses:
      '200':
        description: A list of accounts.
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Account
```

If-Match and If-None-Match

The API that supports updates to resources using optimistic concurrency control must support the If-Match header. Use If-Match and If-None-Match headers with ETag values to check whether the current version of a resource matches the one last retrieved. This is used by the server to verify a precondition set by the ETag before completing the request's processing mechanism and giving a response. Services may also use other headers related to ETags as long as they follow the HTTP specification.

The If-None-Match header can be used to make a request conditional. This allows a cache to update resources only if they have changed.

© 2025 – All rights reserved. ISO 20022 Registration Management Group

Header Element	Usage Guidelines	Usage
Accept	Must	Indicates which content types, expressed as MIME types, the client is able to understand.
Accept-Encoding	Conditional	Indicates the content encoding (usually a compression algorithm) that the client can understand.
Authorisation	Must	Contains the credentials. The element "Authorisation" is implicitly defined by the OAS tag "security".
If-Match	Conditional	Conditional information to support concurrency control or to support caching.
If-None-Match	Conditional	Conditional information to support concurrency control or to support caching.

Standard response headers

ETag

The API should support the ETag header in the GET HTTP response where it is reasonable for clients or proxies to cache the associated resource representation. Where caching is not appropriate, services must include a Cache-Control header and must not include an ETag header. Where ETag is supported, such resources should also support If-Match and If-None-Match headers. PUT, PATCH or DELETE should return HTTP response code 412 if the resource version has changed.

Header Element	Usage Guidelines	Usage
Content-Encoding	Must	The header is used to compress the media-type and tells the client know how to decode it. GZIP or DEFLATE, as appropriate
Content-Type	Must	Indicates the actual media type of the returned content by the service provider.
ETag	Conditional	The ETag response-header element provides the current version of the resource. Used with If-Match and If-None-Match to implement optimistic concurrency control.
Location	Conditional	Points to a newly created resource or to a redirect URL. Other than redirect, use should be limited to return the location of a new resource in the response method of POST/PUT with response code 201.

6.4 Naming conventions

To ensure uniform design, naming must be consistent across the API portfolio. Designing consistent APIs starts with choosing standardised names for every aspect of the API definitions.

To provide a consistent developer experience across the API portfolio, all names used by an API must be consistent, including (query/path/header) parameters, paths, resources, request/responses objects and their fields data type or format.

Naming is a stylistic choice; below are common naming patterns and guidelines for consistent and predictable API design.

 © 2025 – All rights reserved. ISO 20022 Registration Management Group

6.4.1 Resource names

All resource names defined by an API must be unique within the API. Resource names must follow the URI path schema, i.e., they should be in kebab-case. Resource names must be nouns. Collection resource names should be plural nouns.

Examples

Resource name	Usage
/accounts	The collection of all accounts
/accounts/{account-identification}	The single instance of a resource within a collection (here a specific account).

6.4.2 Properties (fields)

Properties (or fields) are the attributes that describe the data model of inputs (request body) and outputs (response). The name of a property should either be camelCase or snake_case. In the context of ISO 20022, they should follow the format defined in the ISO 20022 information model (see <<ref to other document>>).

It is good practice is to avoid any prepositions (e.g., “of”, “for”). Thus, instead of “date_of_issuance” (or “dateOfIssuance”), use “issuance_date” (or “issuanceDate”).

6.4.3 Query parameters

A query parameter name should be either camelCase or snake_case, consistent with the case standard employed for property (field) names.

6.4.4 Path parameters

A path parameter is like a variable name and thus its naming format is actually not relevant. Nonetheless, for the sake of homogeneity, it is recommended to use URL style, that is kebab-case.

6.4.5 Header parameters

Header parameters (in request or response) names are case-insensitive. It is recommended to use kebab-case (URL-style).

© 2025 – All rights reserved. ISO 20022 Registration Management Group

7 Version control

An API must be versioned. There are two aspects:

1. The versioning of the specification
2. The version of the API (runtime)

It is recommended to version its specification (i.e., its OpenAPI specification, under the tag info.version) in accordance with semantic versioning (SemVer 2), with three levels MAJOR.MINOR.PATCH. The rules for incrementation are as follows:

- Increment the MAJOR when you make backwards-incompatible API changes.
- Increment the MINOR when you add functionality in a backwards-compatible manner.
- Increment the PATCH when you make backwards-compatible bug fixes.

In case of backwards-compatibility breaks, old and new versions of the API must be running concurrently, at least for some time until all clients have migrated to the latest version. Within a major version, only the latest API needs to be deployed as they are backwards-compatible. Therefore, the major version of the API must be declared for runtime use. There are multiple ways to implement this in practice, nonetheless the most recommended approach is to use URL-based versioning, where the major of the full specification version is included in the URL, with a "v" as prefix, between the {API name} and the {resource-path}, as follows:

https://.../<api-name>/v<major-version>/<resource-path>

Examples:

https://api.my-domain.com/cash-management/v1/accounts
https://api.my-domain.com/cash-management/v1/accounts/{account-identification}

© 2025 – All rights reserved. ISO 20022 Registration Management Group

8 API Security

An API must be properly secured using industry standards.

Communications must be encrypted using TLS 1.2 or above.

There are several widely used authorisation methods for REST APIs, such as:

- HTTP authentication schemes, typically Basic and Bearer
- API Keys
- OAuth 2
- OpenID Connect.

It is recommended to use OAuth 2 or OpenID Connect. JSON Web Token (JWT) is an industry standard (RFC 7519) that is widely used to exchange claims securely between client and server, in particular when used in conjunction with JSON Web Signature (RFC 7515).

Authorisation (typically based on the claims contained in a JWT) are best left to the server (business logic), at least the detailed authorisation.

The following is a non-exhaustive list of additional resources to help secure REST APIs:

- Guidelines from the OWASP (Open Worldwide Application Security Project) Foundation, in particular their OWASP Top 10 API Security Risks.
- Financial-grade API Security Profile Baseline and Advanced

© 2025 – All rights reserved. ISO 20022 Registration Management Group

9 JSON Schema transformation rules

Please refer to ISO 20022 generating JSON Schema for details.

Bibliography

[1] http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

[2] http://restcookbook.com/Basics/hateoas

[4] RFC 7396, JSON Merge Patch, https://tools.ietf.org/html/rfc7396

[5] RFC 6902, JavaScript Object Notation (JSON) Patch, https://tools.ietf.org/html/rfc6902

[6] RFC 8288, Web Linking, https://datatracker.ietf.org/doc/html/rfc8288

[7] RFC 7231, Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content, https://datatracker.ietf.org/doc/html/rfc7231

[8] RFC 3986, Uniform Resource Identifier (URI): Generic Syntax, https://datatracker.ietf.org/doc/html/rfc3986

[9] RFC 9110, HTTP Semantics, https://datatracker.ietf.org/doc/html/rfc9110

[10] RFC 7515, JSON Web Signature (JWS), https://datatracker.ietf.org/doc/html/rfc7515

[11] RFC 7519, JSON Web Token (JWT), https://datatracker.ietf.org/doc/html/rfc7519

[12] RFC 7807, Problem Details for HTTP APIs, https://datatracker.ietf.org/doc/html/rfc7807

[13] OWASP Foundation, https://owasp.org/

[14] Financial-grade API Security Profile 1.0 – Part 1: Baseline, https://openid.net/specs/openid-financial-api-part-1-1_0.html

[15] Financial-grade API Security Profile 1.0 - Part 2: Advanced, https://openid.net/specs/openid-financial-api-part-2-1_0.html

 © 2025 – All rights reserved. ISO 20022 Registration Management Group

10 Revision history

Version	Date	Author	Description	Sections affected
1.0	29 January 2018	ISO 20022 TSG / RMG Editor: Claude Wanner (Pay.UK)	First version published	All
2.0	10 June 2025	ISO 20022 TSG / RMG Editors: Tapas Bhattacharya (Swift), Christian Frei (SIX), Paul Zanotelli (SIX)	Substantial revisions to incorporate Web APIs	All

Disclaimer:

Although the TSG, the RMG and the Registration Authority have used all reasonable efforts to ensure accuracy of the contents of the website of iso20022.org and the information published thereon, the TSG, the RMG and the Registration Authority assume no liability whatsoever for any inadvertent errors or omissions that may appear thereon. Moreover, the information is provided on an "as is" basis. The TSG, the RMG and the Registration Authority disclaim all warranties and conditions, either express or implied, including but not limited to implied warranties of merchantability, title, non-infringement and fitness for a particular purpose.

The TSG, the RMG and the Registration Authority shall not be liable for any direct, indirect, special or consequential damages arising out of the use of the information published on the website of iso20022.org, even if the TSG, the RMG and the Registration Authority have been advised of the possibility of such damages.

Intellectual Property Rights:

The ISO 20022 MessageDefinitions described in this document were contributed by Swift, IFX, OAGi & TWIST as part of ISTH. The ISO 20022 IPR policy is available at www.ISO20022.org > About ISO 20022 > Intellectual Property Rights.

© 2025 – All rights reserved. ISO 20022 Registration Management Group